KU Leuven

Project Report

Bicycle Traffic Monitoring and Forecasting

Modern Data Analytics

Group 16

Ghasemian Moghaddam Alireza	r1079987
Xuanang Li	r1070796
Yavuz Tok	r0882205
Zeren Su	r1082963

Academic year 2025–2026

1 Executive Summary

This project builds a station-level bicycle traffic monitoring and forecasting dashboard for Flanders using the public AWV *fietstellingen* dataset. The intended users are transport planners and infrastructure managers who need to inspect cycling demand, understand local station patterns, identify unusual recent behavior, and access short-term forecasts.

The implementation is deliberately organized as a small modern data analytics system rather than a one-off analysis. Raw monthly CSV files are ingested, cyclist records are filtered and aggregated to hourly station counts, processed data is stored as Parquet, forecast models are evaluated through rolling backtests, and the Shiny dashboard reads saved artifacts instead of retraining models live. This directly responds to the proposal feedback that the project should make the pipeline, retraining, evaluation, and decision-making value more explicit.

The current processed dataset covers 81 monthly files from August 2019 to April 2026. It contains 5,311,003 hourly station records from 150 stations and 107,385,663 counted cyclists. Data coverage is high overall: mean station coverage is 98.86% and median station coverage is 99.58%. Forecasts are generated for all 147 eligible stations that satisfy the minimum history and coverage rules.

The dashboard contains five pages: Overview, Station Detail, Forecast, Planning Insights, and ML Engineering. The planning page separates three questions instead of hiding them inside one score: which stations show consistently strengthening demand, which show consistently weakening demand, and which have many unusual daily counts. The ML Engineering page documents the end-to-end pipeline, generated artifacts, run status, and operational commands, making the project reproducible for teammates and assessors.

2 Problem Context and Objectives

AWV's bicycle counting stations provide a valuable source for understanding cycling demand across fixed locations in Flanders. These stations do not describe every route in the cycling network, but they do provide repeated time series that can support monitoring and planning. The project therefore focuses on station-level decision support rather than individual cyclist routing.

The main project objectives are:

- process large monthly bicycle count files into fast dashboard-ready datasets;
- monitor all processed bicycle counting stations through maps, rankings, and station detail views;
- generate short-term hourly forecasts for eligible stations;
- evaluate models against a strong seasonal baseline using rolling time-based backtests;
- turn recent traffic patterns into planning signals that are easy to explain;
- document how the system can be rerun when new data is released.

The stakeholder value is strongest for infrastructure supervisors and mobility planners. They can use the dashboard to screen locations that deserve further investigation, not to make automatic investment decisions. The dashboard highlights where demand is high, where recent demand is strengthening or weakening, and where data quality or unusual counts may require closer checking.

3 Data Source and Preprocessing

3.1 Input Data

The project uses the public AWV bicycle count dataset. The raw source contains monthly count files and metadata files. The monthly count files include station IDs, direction, object type, start and end timestamps, and counts. The station metadata provides names, municipalities, coordinates, and installation-related information.

Table 1: Main raw data components.

Input	Use in this project
`data-YYYY-MM.csv`	Monthly count files. The pipeline keeps `FIETSERS` records and aggregates them to hourly station totals.
`sites.csv`	Station names, municipalities, coordinates, and labels used in maps and tables.
`richtingen.csv`	Direction metadata. The current dashboard aggregates directions to station totals for clearer planning interpretation.

3.2 Preprocessing Pipeline

The preprocessing script reads monthly files in chunks to avoid loading the full raw dataset into memory. It filters to cyclist observations, parses timestamps, floors observations to hourly resolution, aggregates by `site_id` and hour, merges station metadata, and creates station-level summaries.

The dashboard does not read the raw CSV files directly. Instead, the pipeline writes reusable artifacts to `data/processed/`. This separation keeps the dashboard fast and makes the system easier to rerun and debug.

Table 2: Current processed dataset summary.

Processed data statistic	Value
Monthly files processed	81
Date range	2019-08-01 to 2026-04-30
Hourly station records	5,311,003
Stations with counts	150
Total counted cyclists	107,385,663
Mean station coverage	98.86%
Median station coverage	99.58%

Station data coverage is computed as:

$$\text{coverage rate} = \frac{\text{observed hourly records}}{\text{expected hourly records between first and last observation}}.$$

A zero count can still be a valid observed hour; coverage measures whether the hour exists in the data, not whether bicycles passed the station. Coverage is used as a quality safeguard, especially for training eligibility and planning trend views.

3.3 Station-Level Features and Quality Safeguards

The processed station summary is used by several dashboard pages. It contains descriptive traffic indicators, quality indicators, and metadata. The most important features are average daily traffic, peak hour count, 95th percentile hourly count, active days, data coverage, recent and previous average daily traffic, and station coordinates.

Several of these features are deliberately simple. For example, average daily traffic is computed

using the number of active days for that station rather than the global number of days in the whole dataset. This is important because stations do not all have the same active history. If a station was installed later, dividing by the full global date range would underestimate its typical daily use.

Table 3: Main station-level features and how they support dashboard interpretation.

Feature	**Interpretation in the dashboard**
Average daily traffic	Typical daily demand for a station over the selected period, using station-specific active days.
Peak hour count	The maximum observed hourly count for the selected station and period.
95th percentile hourly count	A robust high-demand indicator that is less sensitive than the single maximum.
Coverage rate	Completeness of hourly observations between a station's first and last observed hour.
Moving averages	1-week, 2-week, and 4-week average daily counts used for trend screening.
Robust weekday baseline	Median and MAD-based scale by station and weekday, used for daily outlier detection.

The quality safeguards are intentionally visible rather than hidden. Low coverage can affect growth, decline, and forecast reliability. For this reason, the forecasting pipeline requires sufficient historical coverage and active days, while the planning trend view requires recent 4-week coverage of at least 80%. This does not remove all possible data issues, but it prevents the most obvious missing-data cases from being presented as strong planning signals.

4 System Architecture and Reproducibility

The project is organized around a repeatable pipeline. The raw data folder and generated artifacts are not uploaded to GitHub because they are large and reproducible. Source code, report files, configuration, and documentation remain version-controlled.

Table 4: Generated artifacts consumed by the dashboard.

Artifact	**Role**
`hourly_counts.parquet`	Hourly station-level cyclist counts used by all dashboard pages.
`station_summary.parquet`	Station metadata, coverage, average traffic, and summary statistics.
`model_evaluation.csv`	Average model metrics across rolling backtest windows.
`model_backtest_windows.csv`	Detailed per-window model evaluation results.
`forecast_outputs.parquet`	Saved 168-hour future forecasts for trained stations.
`pipeline_run_summary.json`	Metadata for preprocessing runs.
`forecast_run_summary.json`	Metadata for forecast training and evaluation runs.
`pipeline_orchestration.json`	Metadata for one-command pipeline execution.

The end-to-end flow is:

Ingest raw data	Build features	Train and validate	Serve dashboard
Monthly AWV CSV files	Hourly counts and station summaries	Rolling backtests and forecasts	Shiny reads saved artifacts

The command-line runner `codes/run_pipeline.py` supports both full refreshes and forecast-only refreshes. This is important because AWV releases new monthly data over time. The

current development workflow runs locally with Python dependencies from `requirements.txt`. A Dockerfile is included for reproducibility and future deployment, but Docker is not required to run the current project locally.

4.1 Collaboration and Local Execution

The project is designed so teammates can reproduce it without committing large data files to GitHub. The repository contains the source code, dashboard, report, dependency file, and documentation. The raw data folder `fietstellingen_csv/` and the generated `data/processed/` artifacts are intentionally excluded through `.gitignore`. This keeps the GitHub repository lightweight and avoids uploading large reproducible data files.

There are two practical ways for a teammate to run the project. If they have the raw AWV CSV files, they can place `fietstellingen_csv/` in the project root and run the full pipeline. If they receive the processed artifacts separately, they can place `data/processed/` in the project root and start the dashboard directly. The second option is faster for demonstration, while the first option is more reproducible.

4.2 Course Alignment

The implementation connects several parts of the Modern Data Analytics course. The goal was not only to build a visual dashboard, but to show how data processing, modeling, dashboarding, versioning, and infrastructure thinking fit together.

Table 5: Relationship between course concepts and the final project.

Course topic	**How it appears in the project**
Python and packages	Pandas, NumPy, Plotly, Shiny, and scikit-learn are used in a structured Python project.
Data science algorithms	Aggregation, ranking, moving averages, robust outlier detection, and forecast metrics are implemented.
Scikit-learn pipelines	Histogram gradient boosting is wrapped with median imputation and engineered time-series features.
Dashboarding in Python	Shiny for Python provides interactive pages, station selection, tables, maps, and plots.
Code collaboration	GitHub tracks code and documentation, while `.gitignore` protects large data artifacts.
Containerisation and cloud	Docker support is prepared, and the forecast step can later be moved to scheduled cloud jobs.
Analytics infrastructure	The ML Engineering page exposes pipeline stages, artifacts, run status, and re-run commands.

5 Forecasting Methodology and Results

5.1 Forecasting Task

The forecasting task is short-term hourly prediction at station level. For a selected station, the dashboard can show the next 24 hours or the next 7 days hourly. The saved forecast artifact contains 168 hours, so the 24-hour view is a filtered view of the same forecast output.

Forecasts are generated for all 147 eligible stations. A station is eligible if it satisfies the minimum data coverage rule and has enough active days for training. The remaining stations are still available in monitoring pages, but they are not used for saved forecast outputs because their history or quality is insufficient.

5.2 Models and Features

The forecasting pipeline compares two models:

- **Seasonal naive**: predicts a future hour using the same hour from the previous week, with fallback hour-of-week medians.
- **Histogram gradient boosting**: a scikit-learn model using calendar features, lag features, and rolling statistics.

The machine learning model uses only internal count history. This keeps the project lightweight and avoids the extra reliability risks of external data such as weather, holidays, or road works. The engineered features include cyclic hour, weekday, and month features; a weekend indicator; lags at 1, 24, and 168 hours; rolling means over 24 and 168 hours; and rolling 24-hour standard deviation.

Table 6: Rationale for the current forecasting design.

Modeling choice	**Reason**
Seasonal naive baseline	Weekly cycling patterns are strong, so a simple previous-week baseline is hard to beat and gives a fair reference.
Histogram gradient boosting	Captures nonlinear relationships between calendar features, recent lags, and rolling statistics without heavy dependencies.
No external weather data yet	Keeps the first system reliable and reproducible; external sources can be added once the internal pipeline is stable.
Per-station training	Stations differ in volume and behavior, so each station receives its own model evaluation and best-model decision.

5.3 Rolling Backtests

The evaluation uses three rolling backtest windows. Each window trains on the latest available history before the test period and evaluates a 168-hour held-out week. The windows are spaced 168 hours apart. The current training window length is 728 days. Metrics are averaged per station and model.

The main evaluation metric is WAPE:

$$\text{WAPE} = \frac{\sum_t |y_t - \hat{y}_t|}{\sum_t y_t} \times 100.$$

WAPE is useful because it scales absolute error by total observed demand. However, very low-volume stations can produce extreme WAPE values, so medians and interquartile ranges are more informative than means for the all-station run.

This setup avoids random cross-validation because the data is time ordered. Random folds would leak future observations into training and make the forecast evaluation too optimistic. Rolling backtests are more appropriate because each test week is predicted using only historical data that would have been available at that time.

5.4 Current Forecast Results

The latest forecast run uses 147 eligible stations, 2 candidate models, 3 rolling backtest windows, and a 168-hour forecast horizon. It produces 882 detailed backtest rows, 294 aggregated evaluation rows, and 49,392 forecast rows.

Histogram gradient boosting is the best model for most eligible stations, but the seasonal naive baseline still wins for 18 stations. This confirms that the baseline is meaningful: bicycle traffic often follows a strong weekly pattern, and a more complex model should only be preferred when it improves on that pattern. The dashboard therefore selects the best model per station based on average WAPE rather than assuming one universal best model.

Table 7: Forecast model performance across the 147 eligible stations.

Model	Median WAPE	IQR WAPE	Best for stations
Histogram gradient boosting	34.16%	28.89–40.92%	129
Seasonal naive	39.85%	33.92–44.89%	18

6 Dashboard Design

The dashboard is implemented with Shiny for Python and Plotly. It contains five pages that move from monitoring to station analysis, forecasting, planning signals, and engineering transparency.

Table 8: Dashboard pages and their decision-support role.

Dashboard page	Main question answered
Overview	Where are the stations and which locations have the highest average daily traffic?
Station Detail	What is the historical pattern of one selected station?
Forecast	What are the expected hourly counts for the next 24 hours or 7 days, and how reliable are the models?
Planning Insights	Which stations show strengthening demand, weakening demand, or unusual recent daily counts?
ML Engineering	How were the data, models, forecasts, and dashboard artifacts produced and how can they be rerun?

6.1 Overview

The Overview page summarizes the full processed dataset for a selected date range. It shows the number of stations, hourly rows, total cyclists, a station map, and a ranking of highest average daily traffic. The map marker size and color use the same definition as the ranking table: total count divided by the number of active days for that station in the selected period. This avoids misleading comparisons when stations have different active histories.

6.2 Station Detail

The Station Detail page focuses on one selected station. It reports average daily traffic, peak hour count, and data coverage. It also shows three plots: daily traffic over time, average pattern by hour of day, and weekday profile. These views help planners understand whether a station is mainly commuter-oriented, weekend-oriented, seasonal, or affected by missing data.

6.3 Forecast

The Forecast page shows saved future forecasts for the selected station. It includes three value boxes: baseline WAPE, best ML WAPE, and best model. The main plot combines recent history with seasonal naive and histogram gradient boosting forecasts. A rolling-backtest table reports MAE, RMSE, WAPE, number of backtest windows, and test period. The scope and interpretation text at the bottom explains that monitoring uses all 150 stations while saved forecasts currently cover all 147 eligible stations.

6.4 Planning Insights

The Planning Insights page separates planning signals into three focused views so that planners can inspect one question at a time.

- **Fast Growth**: uses ma_1w > ma_2w > ma_4w, signaling strengthening demand.
- **Fast Decline**: uses ma_1w < ma_2w < ma_4w, signaling weakening demand.

- **Daily Outliers**: flags unusual daily counts against each station's own same-weekday history.

The moving-average rules are inspired by the idea of bullish and bearish moving-average alignment in financial charts, but they are used here only as monitoring signals. The 1-week, 2-week, and 4-week average daily counts make the trend easy to inspect directly from the table. In the latest run, 127 stations satisfy the fast-growth alignment and 3 stations satisfy the stricter fast-decline alignment.

The Daily Outliers tab uses robust weekday-specific statistics. For each station and weekday, the system estimates a historical median and a MAD-based robust scale. A day is marked as an outlier when its absolute robust z-score exceeds 2.576, corresponding to a two-sided 99% cutoff under a normal approximation. The table ranks stations by the number of outlier days in the latest four weeks.

Table 9: Examples of planning signals produced by the current dashboard.

Signal	Example station	MA 1w	MA 4w	Interpretation
Fast Growth	Bertem	293.71	186.46	Short-term demand is clearly above the broader 4-week level.
Fast Growth	Bree	969.14	667.43	Recent demand is strengthening relative to the recent month.
Fast Decline	Tienen 2	0.43	413.36	Very strong decline signal; should be checked for context or sensor issues.
Daily Outliers	Tervuren	–	–	18 outlier days in the latest four weeks under the 99% robust cutoff.

These signals are intentionally separated. Combining them into one score would make the dashboard harder to explain because high growth, low coverage, and outlier behavior mean different things. A planner can first inspect growth, then decline, then unusual days, and decide which interpretation is most relevant for the local context.

6.5 ML Engineering

The ML Engineering page makes the system transparent. It starts with an end-to-end pipeline view: ingest raw data, build features, train and validate, and serve the dashboard. It then shows processed rows, forecast station coverage, backtest windows, and forecast horizon. The run-status table checks preprocessing, forecast training, orchestration, and dashboard serving. The artifact table groups outputs into features, model validation, forecast serving, and run metadata. Finally, operational commands show how to rerun the full pipeline, retrain forecasts only, or launch the dashboard.

This page is included because it makes the engineering work visible. Without it, the dashboard could look like a collection of plots. With it, the user can see which artifacts exist, when the pipeline was last run, how many stations were forecasted, and which commands regenerate the outputs. This is especially useful for a course project because it demonstrates that the system can be maintained and extended after the first submission.

7 Response to Feedback and Course Alignment

The proposal feedback asked for a stronger ML engineering component and clearer decision-making value for the dashboard. The final implementation addresses this in several ways.

Table 10: How the implementation responds to proposal feedback.

Feedback point	Implemented response
Dashboard too limited for infrastructure planning	Added separate fast-growth, fast-decline, and daily-outlier planning views.
Risk of one-off analysis	Added preprocessing, forecast training, orchestration scripts, saved artifacts, and run metadata.
Need retraining and evaluation over time	Added rolling backtests and commands for monthly refreshes.
Forecasting setup too standard	Compared a strong seasonal naive baseline with histogram gradient boosting and selected the best model per station.
Need explicit decision-making value	Used interpretable moving-average rules and robust same-weekday outlier detection as planning screening tools.

The project also connects directly to the Modern Data Analytics course. Python and pandas are used for data processing, scikit-learn is used for feature-based forecasting, Shiny for Python is used for dashboarding, GitHub is used for collaboration and versioning, and the Dockerfile prepares the project for standardized execution even though local development remains the main workflow.

8 Limitations and Future Work

The dashboard should be interpreted as a screening and monitoring tool. It highlights stations that deserve attention, but it does not prove causal explanations. For example, fast growth may be caused by seasonality, infrastructure changes, weather, events, or actual demand shifts. Similarly, a daily outlier signal means the count is unusual relative to the station's same-weekday history, but it does not automatically mean the sensor is broken.

The current forecasting model uses only historical bicycle counts. Weather, holidays, school vacation periods, road works, and local events may improve forecasting, but they would also add data integration and maintenance complexity. For the current course project, the priority is a reliable internal-data pipeline.

The all-station forecast run is already implemented locally. If future models become heavier, the forecast training step can be moved to cloud infrastructure. A realistic production setup would store raw and processed data in S3, run scheduled containerized training jobs on EC2 or AWS Batch, and write forecast artifacts back to shared storage for the dashboard to consume.

Docker support is prepared through a Dockerfile and `.dockerignore`, but Docker is not required for the current team workflow. The practical current workflow is: clone the GitHub repository, obtain raw or processed data separately, install dependencies from `requirements.txt`, rerun the pipeline if needed, and start the Shiny dashboard. Docker should be treated as a reproducibility and future deployment option, not as a requirement for local use.

9 Conclusion

This project delivers a working station-level bicycle traffic monitoring and forecasting dashboard backed by a reproducible data and ML pipeline. It processes more than 5.3 million hourly records from 150 stations, generates saved forecasts for 147 eligible stations, compares a strong seasonal naive baseline with histogram gradient boosting, and exposes planning signals through an interactive Shiny dashboard.

The main contribution is the combination of dashboarding and ML engineering. The dashboard is not only a visualization layer; it is supported by preprocessing scripts, Parquet artifacts, rolling backtests, forecast outputs, run metadata, and operational commands. This makes the project closer to a maintainable analytics system than to a one-time analysis, and it directly reflects the Modern Data Analytics themes of Python data science, machine learning pipelines, dashboarding, reproducibility, and analytics infrastructure.

A Reproducibility Commands

The raw data folder `fietstellingen_csv/` and generated `data/processed/` files are ignored by Git. They can be regenerated locally from the downloaded AWV CSV files.

```
# Full refresh from raw CSV files
python codes/run_pipeline.py \
  --start-month 2019-08 \
  --end-month 2026-04 \
  --all-stations \
  --backtest-windows 3

# Forecast-only refresh if processed data already exists
python codes/run_pipeline.py \
  --skip-preprocess \
  --all-stations \
  --backtest-windows 3

# Run the dashboard locally
shiny run --host 127.0.0.1 --port 8000 --launch-browser app/app.py
```

B Main Project Files

File	Purpose
`codes/preprocess.py`	Builds hourly counts and station summaries from raw CSV files.
`codes/train_forecasts.py`	Runs rolling backtests and generates future forecasts.
`codes/run_pipeline.py`	Orchestrates preprocessing and forecast training.
`app/app.py`	Shiny dashboard application.
`app/www/styles.css`	Dashboard styling.
`requirements.txt`	Python dependencies for local execution.
`Dockerfile`	Optional container definition for future reproducibility or deployment.
`README.md`	Main setup and running instructions.

C Dashboard Demonstration Checklist

1. Start with **Overview**: map, total cyclists, station count, and highest daily traffic ranking.
2. Move to **Station Detail**: average daily traffic, peak hour count, coverage, daily trend, hourly pattern, and weekday profile.
3. Show **Forecast**: forecast horizon, baseline WAPE, best ML WAPE, best model, rolling backtest table, and scope explanation.
4. Explain **Planning Insights**: fast growth, fast decline, and daily outliers as three separate planning questions.
5. End with **ML Engineering**: end-to-end pipeline, run status, artifacts, and operational commands.

D Docker Note

The project includes Docker support, but Docker is not necessary for the current local workflow. If Docker Desktop or another Docker runtime is installed, the intended commands are:

```
docker build -t mda-bike-dashboard .
docker run --rm -p 8000:8000 \
  -v "$PWD/data/processed:/app/data/processed" \
  mda-bike-dashboard
```

During the latest local check, Docker CLI was not available on the development machine. Therefore, Docker should be described as prepared for reproducibility and future deployment, not as the main verified execution path.